1 Syllabus covered

1.1 Combinational logic

- ☑ Generate minterms, maxterms, SOP canonical form and POS canonical forms and convert between them
- ☑ Understand and use the laws and theorems of Boolean Algebra
- ☑ Perform algebraic simplification using Boolean algebra
- ☑ Derive sum of product and product of sums expressions for a combinational circuit
- ☑ Convert combinational logic to NAND-NAND and NOR-NOR forms
- ☑ Simplification using K-maps

1.2 Sequential logic

- ☑ Understand the difference between synchronous and asynchronous inputs
- ☑ Derive a state graph or state table from a word description of the problem
- ☑ Implement a design using JK, SR, D or T flip-flops
- ☑ Design a sequential circuit and derive a state-table and a state-graph
- ☑ Design and design both Mealy and Moore sequential circuits with multiple inputs and multiple outputs
- ☑ Reduce the number of states in a state table using row reduction and implication tables
- ☑ Perform a state assignment using the guideline method

1.3 Discrete Mathematics

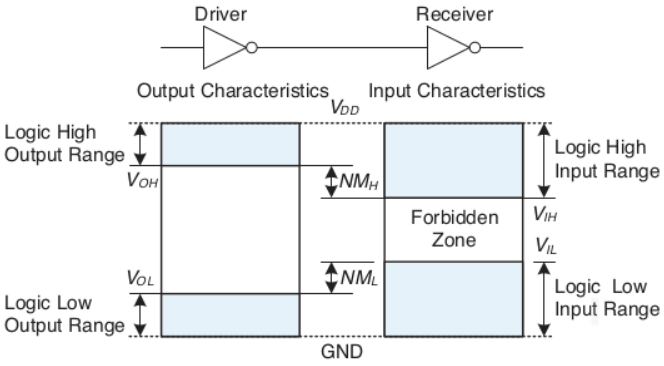
- ☑ Defining Propositional Logic.
- ☑ Defining Predicate Logic.
- ☑ Defining Set, function, or relation model, and interpret the associated operations.
- ☑ Operations between sets, functions, and relations.
- ☑ Constructing proofs.

- ☑ Logical reasoning to solve a strategic problem.
- ☑ Model a real-life industry application applying formal methods.
- ☑ Design combinational circuits using multiplexers and decoders
- ☑ Different between and limitations of level-triggered latches and edge-triggered flip-flops.

1.4 Labs (not asked in final exam)

- ☑ Quartus/ModelSim installation and setup to interface it with Cyclone III FPGA.
- ☑ Create a multiplexer using the GUI schematic functionality in Quartus
- ☑ Create a multiplexer with SystemVerilog only using the NOT, AND, OR operators.
- ☑ Compare the gates synthesized from your SystemVerilog code by Quartus to the schematic design
- ☑ Extend the multiplexer to accept a vector of inputs
- ☑ Write and use SystemVerilog modules for reuse the modules
- ☑ Learn to write state machines using Verilog
- ☑ Learn about Behavioral Verilog and the following Verilog syntax
  - ☑ Number representation: 3'b010, 4'd4, 8'hFF
  - ☑ Concatenation `{SW[1:3], SW[4:7]}`
  - ☑ Conditional operator f = x ? y : z;
  - ☑ Always block: `always_comb`, `always_ff` @(Sensitity list)
  - ☑ case statement
  - ☑ if statement
  - ☑ logic type in System Verilog, vs wire and reg type in Verilog
  - ☑ posedge keyword
  - ☑ 'include keyword
  - ☑ Synthesizer directives: /* synthesis keep */
  - ☑ Non-synthesizable statements for simulation: `$display, 'timescale`

2 Logic levels and Noise Margins



Definition 1 (Input high (V_{IH}) and Input Low (V_{IL}) of a gate) *V_{IH} is the voltage level, such that an input voltage to a gate between V_{DD} and V_{IH} is considered* HIGH. *Similarly, input voltage to a gate between V_{IL} and V_{GND} is considered* LOW.

Definition 2 (Output high (V_{OH}) and Output low (V_{OL}) of gate) *V_{OH} is the voltage level, such that an output voltage to a gate between V_{DD} and V_{OH} is considered* HIGH. *Similarly, output voltage to a gate between V_{OL} and V_{GND} is considered* LOW.

Definition 3 (Positive logic and Negative logic) *What we have considered so far is Positive logic where* HIGH *voltage is equated to Boolean logic* TRUE *or* 1 *and* LOW *is considered* FALSE *or* 0. *In negative logic these are reversed. Same physical circuit can represent different logical circuits in positive logic and negative logic.*

Definition 4 (Noise margins (NM_L and NM_H) of a channel) *The maximum amount of noise that can be added (or substracted) to a channel without exceeding the logic level specifications of a gate.* $NM_L = V_{IL} - V_{OL}$
$NM_H = V_{OH} - V_{IH}$

Example 1
If $V_{DD} = 5V$, $V_{IL} = 1.35V$, $V_{IH} = 3.15V$, $V_{OL} = 0.33V$ and $V_{OH} = 3.84V$ for both the "inverters", then what are the low and high noise margins? Can the circuit tolerate 1V of noise at the channel?

Example 2 *Design a T-ff using J-K ff*